Comprehensive Guide to Numerical Integration using NumPy

Data Science & Applied Mathematics Notes

August 14, 2026

1 Introduction and Background

Numerical integration is a fundamental computational technique used to approximate the definite integral of a function:

$$I = \int_a^b f(x)\,dx \tag{1}$$

This approach is essential in real-world applications when:

- The analytical antiderivative of $f(x)$ is difficult or impossible to find (e.g., e^{-x^2}).
- The function $f(x)$ is not explicitly known, and we only possess discrete data samples from experiments or sensors.

Instead of finding an exact algebraic solution, numerical integration splits the area under the curve into small geometric regions, calculates the area of each distinct shape, and aggregates them.

—

2 A Simple Mathematical Walkthrough

To understand the process without complex code, we can approximate the area under the curve $y = x^2$ from $x = 0$ to $x = 2$ using basic geometric approximations.

2.1 The Rectangle Approximation Method

If we split the domain into 2 equal subdivisions ($n = 2$), the width (Δx) of each slice is calculated as:

$$\Delta x = \frac{b-a}{n} = \frac{2-0}{2} = 1 \tag{2}$$

Our discrete boundary evaluation points sit at $x_0 = 0$, $x_1 = 1$, and $x_2 = 2$.

Using a **Right Riemann Sum**, we define the height of each rectangle by the evaluation point on its right-hand edge:

- **Rectangle 1 ($x \in [0, 1]$):** Height evaluated at $x = 1 \implies 1^2 = 1$.
- **Rectangle 2 ($x \in [1, 2]$):** Height evaluated at $x = 2 \implies 2^2 = 4$.

Summing these individual sections yields:

$$\text{Approximate Area} = (\text{Width} \times \text{Height}_1) + (\text{Width} \times \text{Height}_2) = (1 \times 1) + (1 \times 4) = 5 \tag{3}$$

Analytical Comparison: The true calculus area is exactly $\int_0^2 x^2 dx = \frac{8}{3} \approx 2.67$. The rectangle approximation overestimates because it contains large empty blocks above the true curve.

—

3 The Trapezoidal Rule Architecture

The Trapezoidal Rule reduces approximation error by replacing flat-topped rectangles with sloped trapezoids that closely mimic the curve's trajectory.

The fundamental mathematical equation for the area of a single trapezoid with width w bounded by vertical heights y_1 and y_2 is:

$$\text{Area} = w \times \frac{y_1 + y_2}{2} \tag{4}$$

—

4 NumPy Vectorized Mechanics

Modern NumPy applications utilize the modern function `numpy.trapezoid`. The legacy alias `numpy.trapz` has been officially deprecated and completely removed from the library core. The backend scales calculations depending on how dimensions are defined:

4.1 Case 1: Uniform Spacing via `dx`

When data coordinates are uniformly distributed, every slice shares a matching width (dx). Summing adjacent trapezoids shares inner boundary calculations (y_1 through y_{n-2} are counted twice), allowing NumPy to collapse the operation into a highly optimized equation:

$$\text{Total Area} = dx \times \left(\frac{y_0}{2} + y_1 + y_2 + \cdots + y_{n-2} + \frac{y_{n-1}}{2} \right) \tag{5}$$

The first and last values are scaled by half, the inner values are weighted fully, and the total sum scales against the dx constant.

4.2 Case 2: Non-Uniform Spacing via explicit `x`

When coordinate locations vary across a variable grid, an absolute global shortcut is mathematically invalid. NumPy dynamically computes localized slice width vectors:

$$w_i = x_{i+1} - x_i \quad \text{(equivalent to } \texttt{np.diff(x)}\text{)} \tag{6}$$

The localized midpoint heights are established simultaneously:

$$h_i = \frac{y_i + y_{i+1}}{2} \tag{7}$$

NumPy then executes a fast vectorized linear dot product to gather total area properties across the arrays:

$$\text{Total Area} = \sum_{i=0}^{n-2} w_i \cdot h_i \tag{8}$$

—

5 Practical Overleaf Ready Implementations

```
import numpy as np

# Define underlying test metrics matching our mathematical examples
# Example A: Constant spacing matrix evaluated at dx = 1.0
y_uniform = np.array([1.0, 4.0, 9.0])
area_uniform = np.trapezoid(y_uniform, dx=1.0)
print(f"Uniform Spacing Calculation Result: {area_uniform}")

# Example B: Variable coordinate tracking matching custom x profiles
x_skewed = np.array([0.0, 1.0, 3.0])
y_skewed = np.array([1.0, 4.0, 10.0])
area_skewed = np.trapezoid(y_skewed, x=x_skewed)
print(f"Variable Coordinate Calculation Result: {area_skewed}")
```

Listing 1: Numerical integration scripts adapting to changing step properties